# Lab 13 — Services and Ingress: exposing

*Theory — why a Pod's IP is useless, how a stable name attaches to Pods that keep changing, and through which doors traffic enters the cluster: ClusterIP, NodePort, LoadBalancer, Ingress.*

## Objectives

- Understand what a **Service** adds to Pods: a virtual IP, a DNS name, load distribution.
- Tell `ClusterIP`, `NodePort` and `LoadBalancer` apart, and know which one to use.
- Read **EndpointSlices** and understand their link with readiness.
- Expose the front end through an **Ingress** and understand the role of the Ingress controller (Traefik).
- Recognise, in all of this, the DNS and reverse proxy of labs 07 and 09.

---

## 1. The problem: IPs that never stop changing

In lab 12, every rollout replaced the Pods, and every new Pod received a new `10.42.0.x` IP. A front end that had hard-coded the address of an API Pod would have fallen over at the first update. Worse: with three replicas, which one to call? In the hands-on lab, an nginx started before any Service fails flat: `host not found in upstream "api"` — nothing in the cluster carries that name.

The **Service** solves both problems: it provides a stable **name** and **IP** (the cluster IP, `10.43.x.x` with k3s, never reassigned as long as the Service exists) in front of a **set of Pods** chosen by label, and spreads connections among them. Pods come and go; the Service stays.

> **→ DOCKER RECAP**
> In lab 07, the user-defined network gave a DNS name to every **container**; in lab 09, Compose did it for every **service**, and `--scale api=2` gave two addresses to the same name. The Kubernetes Service is the mature form of that idea: one name, one virtual IP, a list of real addresses kept up to date by the cluster, and distribution done by the node rather than by the client.

## 2. Anatomy of a Service

```
apiVersion: v1
kind: Service
metadata:
  name: api                 # the DNS name
spec:
  selector:
    app: api                # the targeted Pods, by label
  ports:
    - port: 8080            # Service port (the one clients call)
      targetPort: 8080      # container port
```

Three mechanisms hide behind this file. **DNS**: CoreDNS creates the entry `api.lab13.svc.cluster.local` → cluster IP; from the same namespace, `api` is enough thanks to the `search` line in the Pods' `/etc/resolv.conf`. **EndpointSlices**: a controller continuously lists the IPs of the Pods that carry the label **and** are `Ready`; `kubectl get endpointslices` shows them,

and that is the real list behind the virtual IP. **kube-proxy**: on every node, it programs rules (iptables or nftables) that rewrite any connection to the cluster IP into a connection to one of the endpoints, at random.

> **→ NETWORK**
> The cluster IP is attached to **no interface**: no process listens on it, `ping` gets no answer. It is a purely virtual address that kube-proxy's **NAT** rules transform on the fly into a Pod address, on the node the packet leaves from. Distribution therefore happens per connection, at layer 4 (TCP), with no notion of HTTP request — a client that keeps its connection open stays on the same Pod.

Two practical consequences. A Pod that is **not ready** (readiness failing, lab 12) disappears from the EndpointSlices: the Service stops sending it traffic without killing it — that is the whole point of readiness. And a Service **without a selector** has no endpoints: useful for naming a database outside the cluster (with a hand-written EndpointSlice), a trap when the label is misspelled (`kubectl get endpointslices` empty = first reflex).

## 3. Service types: where traffic comes in

| Type | Reachable from | How | When |
|---|---|---|---|
| **ClusterIP** (default) | inside the cluster | virtual IP + DNS | every internal service: `api`, `db` |
| **NodePort** | outside, through a port on **every node** (30000-32767) | `node-IP:30080` → Service | tests, labs, access without a load balancer |
| **LoadBalancer** | outside, through an IP provided by the infrastructure | an external load balancer → NodePort → Service | public exposure in the cloud |
| **ExternalName** | inside | a DNS alias to an external name | naming a service outside the cluster |

Each type **includes** the previous one: a NodePort also has a ClusterIP, a LoadBalancer also has a NodePort. NodePort shows a limit: you need to know a node's address, and the port is odd. LoadBalancer fixes that by asking the infrastructure for a "proper" address — in the cloud, that is a billed load balancer; on a bare cluster, `EXTERNAL-IP` stays `<pending>` forever, for lack of a provider.

> **K3S**
> k3s ships **ServiceLB** (formerly Klipper): for every `LoadBalancer` Service, it launches a `svclb-<name>` Pod in `kube-system` that opens the requested port **on the node** and relays it; `EXTERNAL-IP` becomes the node's IP. Two Services therefore cannot request the same port (the second stays `<pending>`), which is why Traefik already occupies ports 80 and 443. On EKS/AKS/GKE, the same manifest produces a real load balancer with a public IP: the YAML does not change, the bill does.

> **→ WINDOWS / WSL**
> A NodePort or a ServiceLB port is reachable from Windows at `http://<distribution-ip>:30080` (`hostname -I` in WSL), and most often at `http://localhost:30080` thanks to WSL forwarding — for ports opened by k3s, which runs as root, the restriction on ports < 1024 from lab 07 does not apply: Traefik does listen on **80**. `kubectl port-forward` remains the safest path for a quick test: it depends on no Service type.

## 4. The Ingress: one door for all HTTP

Exposing every front end through a LoadBalancer costs one address (and one load balancer) per application. The **Ingress** pools them: a single HTTP/HTTPS entry point, and **rules** that route by host name (`app.example.com`) and by path (`/api/`) to Services. The Ingress object is only a **description**; an **Ingress controller** — a reverse proxy running in the cluster and reading those objects — does the work: Traefik with k3s, ingress-nginx, HAProxy, or the cloud's application load balancer.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
spec:
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

Traffic then flows: browser → node port 80 (Traefik's ServiceLB) → Traefik Pod → Service `web` → nginx Pod → (for `/api/`) Service `api` → API Pod. Two reverse proxies in a row, Traefik then nginx: it is common, and it is why many teams let the Ingress route `/api/` directly to the `api` Service, removing the proxy from nginx.

**→ HTTP**
The Ingress works at layer 7: it reads the `Host` header and the path of every request, can terminate **TLS** (certificates, often automated by cert-manager), rewrite paths, add headers (`X-Forwarded-For`, precious after lab 07), rate-limit. A Service only sees TCP connections. The Ingress's successor, the **Gateway API**, generalises these rules; Traefik supports it, but the Ingress remains the standard deployed everywhere.

## 5. Diagnosis in four questions

When "it does not answer", walk back up the chain: (1) **are the Pods ready?** `kubectl get pods` — `0/1` = readiness; (2) **does the Service have endpoints?** `kubectl get endpointslices` empty = wrong selector or Pods not ready; (3) **does the name resolve?** `kubectl exec … -- wget -qO- http://api:8080/` from a Pod in the same namespace; (4) **is the entry door the right one?** `kubectl get svc` for type and port, `kubectl get ingress` for address and rules, `kubectl describe ingress` for the backends. A `503` from Traefik almost always means "the targeted Service has no ready endpoint".

## 6. In the workplace

- **One ClusterIP Service per Deployment**, with the same name, in the same manifest: `api` and `db` are never exposed outside the cluster.

- **A single Ingress controller** for the whole cluster, run by the platform team; every application declares its Ingress (or Gateway) rules with a host name and a TLS certificate.
- **The front end calls the API through a relative path** (`/api/`), routed by the Ingress or by nginx: same origin, no CORS (lab 07), and the URL does not change from one environment to the next.
- **NodePorts** stay on the workstation and in labs; `LoadBalancer` is reserved for the rare non-HTTP TCP services (replicated databases, brokers), and billed.

---

## Remember

- A Service = a DNS name + a stable cluster IP + a list of endpoints maintained by label and readiness + distribution by kube-proxy.
- `api` resolves within the namespace, `api.lab13.svc.cluster.local` everywhere; without a Service, nginx dies on `host not found in upstream`.
- ClusterIP for internal use, NodePort for testing, LoadBalancer for an external IP (ServiceLB with k3s, a cloud load balancer elsewhere), Ingress for all HTTP.
- A Pod that is not ready leaves the EndpointSlices; a wrong selector gives a Service with no endpoints and a `503`.
- The Ingress describes rules; the Ingress controller (Traefik) applies them on the cluster's port 80/443.
- Diagnosis: Pods ready → endpoints → DNS → entry door.

## Vocabulary

**Service**: stable name and IP in front of Pods selected by label. — **ClusterIP**: virtual IP internal to the cluster. — **NodePort**: port opened on every node (30000-32767). — **LoadBalancer**: external IP provided by the infrastructure (ServiceLB with k3s). — **EndpointSlice**: list of ready Pod IPs behind a Service. — **kube-proxy**: per-node NAT rules that implement the Service. — **CoreDNS**: the cluster's DNS (`<svc>.<ns>.svc.cluster.local`). — **Ingress**: HTTP routing rules by host and path. — **Ingress controller**: reverse proxy that applies Ingresses (Traefik). — **Gateway API**: the Ingress's successor.